## 📘 SQL Window Functions – Notes

### 🔷 What are Window Functions?

SQL **Window Functions** perform calculations across a set of rows related to the current row **without collapsing the rows** into a single result.

👉 Unlike GROUP BY, they keep individual rows visible.

Used for:

- Aggregations (AVG, SUM, etc.)
- Rankings
- Running totals
- Relative comparisons

---

### 🔷 OVER() Clause

The OVER() clause defines the window (set of rows) for calculation.

It can include:

#### 1️⃣ PARTITION BY

- Divides data into groups.
- Similar to GROUP BY, but does not collapse rows.
- Window function works separately within each partition.

#### 2️⃣ ORDER BY

- Defines order of rows inside each partition.
- Important for ranking and running calculations.

---

### 🔷 Basic Syntax

```
SELECT column_name1,
    window_function(column_name2)
    OVER ([PARTITION BY column_name3]
        [ORDER BY column_name4]) AS new_column
FROM table_name;
```

**Components:**

- window_function → SUM(), AVG(), RANK(), etc.
- PARTITION BY → grouping column

- ORDER BY → sorting column
- new_column → alias for result

---

### 📊 Types of Window Functions

SQL window functions are mainly of **two types**:

---

### 1️⃣ Aggregate Window Functions

These calculate aggregate values over a window but retain individual rows.

**Common Functions:**

- SUM() → Total
- AVG() → Average
- COUNT() → Count rows
- MAX() → Maximum value
- MIN() → Minimum value

---

**Example: Average Salary per Department**

```
SELECT Name, Age, Department, Salary,
    AVG(Salary) OVER(PARTITION BY Department) AS Avg_Salary
FROM employee;
```

✓ Calculates department-wise average
✓ Shows average for every employee in that department

---

### 2️⃣ Ranking Window Functions

Used to rank rows within partitions.

---

### 🔷 RANK()

- Assigns rank based on order.
- Skips ranks if duplicate values exist.

Example:

```
RANK() OVER(PARTITION BY Department ORDER BY Salary DESC)
```

Example Output:

1, 1, 3

(Rank 2 skipped)

---

### 🔷 DENSE_RANK()

- Same rank for duplicates.
- No skipped numbers.

Example Output:

1, 1, 2

---

### 🔷 ROW_NUMBER()

- Assigns unique number to each row.
- No duplicates allowed.
- Always increments by 1.

Example Output:

1, 2, 3

---

### 🔷 PERCENT_RANK()

Shows relative rank between 0 and 1.

**Formula:**

(RANK - 1) / (Total Rows in Partition - 1)

- Highest value → 0
- Lowest value → 1
- Shows relative position

---

### 🔥 Key Differences (Very Important for Interviews)

| Function | Duplicates | Gaps in Rank | Unique Numbers |
|---|---|---|---|
| RANK() | Yes | Yes | No |
| DENSE_RANK() | Yes | No | No |
| ROW_NUMBER() | No | No | Yes |

---

### ⚠️ Common Mistakes & Fixes

#### 1️⃣ No PARTITION BY

- Entire table treated as one group.

#### 2️⃣ Wrong ORDER BY

- Rankings or calculations become incorrect.

#### 3️⃣ Performance Issues

- Can be slow on large datasets.
- Use indexes for optimization.

---

### 💡 Important Concept

👉 GROUP BY reduces rows
👉 Window Functions keep rows

This is the core difference.